Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. ColorOctree marks the combined OctTree row that includes ColorOctree passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	**ColorOctree**	**ADT**	**SoA**	**Fold passes**			**Map passes**		
		fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
OctTree	yes	16	9	0.3774	0.3504	1.08×	0.3088	0.3545	0.87×
Compiler	–	9	8	0.3556	0.0675	**5.27**×	0.1620	0.1793	0.90×
DBQuery	–	15	13	0.1239	0.0972	**1.27**×	0.1407	0.1900	0.74×
DecisionTree	–	7	6	3.632	3.105	**1.17**×	–	–	–
DomTree	–	15	14	0.1401	0.0634	**2.21**×	0.0588	0.0816	0.72×
KDTree	–	17	16	0.1788	0.1431	**1.25**×	–	–	–
LinearListReduction	–	11	11	0.0295	0.0062	**4.74**×	–	–	–
List	–	2	2	0.1296	0.1114	**1.16**×	0.2541	0.3404	0.75×
MonoTree	–	3	2	0.0177	0.0226	0.78×	0.0529	0.0782	0.68×
ObjectGraph	–	5	4	0.0907	0.1240	0.73×	0.1429	0.1622	0.88×
PiecewiseFunctions	–	8	7	0.1382	0.0961	**1.44**×	0.2396	0.2606	0.92×
TernaryTree	–	5	3	0.0280	0.0261	1.07×	0.0834	0.0892	0.94×
Trie	–	10	9	0.0658	0.0348	**1.89**×	0.1511	0.1670	0.91×

Table 2: PAPI speedup summary across all passes. For each program and counter, speedup is AoS/SoA using summed per-pass median counters. >1× means SoA has fewer events.

Program	**CYC**	**INS**	**L1D**	**L1I**	**L2D**	**L2I**	**LLC**
OctTree	1.00×	0.75×	0.80×	0.47×	0.16×	0.62×	**3.66**×
Compiler	**1.21**×	0.52×	**3.56**×	0.81×	**22.34**×	0.43×	**6.56**×
DBQuery	0.95×	0.47×	0.99×	0.48×	**7.56**×	0.36×	**6.09**×
DecisionTree	**1.17**×	0.59×	**17.41**×	**3.17**×	1.04×	**3.35**×	**35.86**×
DomTree	**1.50**×	0.57×	**1.63**×	0.83×	**14.29**×	0.24×	**10.17**×
KDTree	**1.28**×	0.77×	**1.13**×	0.89×	0.27×	0.81×	**4.51**×
LinearListReduction	**7.06**×	0.90×	**34.25**×	**1.63**×	**39.60**×	**2.01**×	**11.37**×
List	0.81×	0.80×	**3.97**×	0.43×	0.11×	0.71×	**1.46**×
MonoTree	0.98×	0.85×	0.50×	0.67×	0.31×	0.54×	**1.23**×
ObjectGraph	0.96×	0.75×	**2.76**×	0.76×	0.34×	0.67×	**2.92**×
PiecewiseFunctions	**1.13**×	0.60×	**2.99**×	0.94×	**2.27**×	0.44×	**4.47**×
TernaryTree	**1.15**×	0.79×	0.37×	0.53×	0.22×	**1.15**×	**1.45**×
Trie	**1.16**×	0.56×	**2.82**×	0.85×	**3.03**×	0.20×	**5.58**×
Geomean	**1.26**×	0.67×	**2.31**×	0.81×	**1.40**×	0.64×	**4.68**×

Table 3: PAPI counter totals across all passes (AoS/SoA). Each entry is the sum of per-pass median counter values.

Program	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
OctTree	2.6e+09/**2.6e+09**	**8.2e+09**/1.1e+10	**1.4e+07**/1.8e+07	**3.8e+05**/8.1e+05	**6.3e+04**/3.9e+05	**1.5e+04**/2.4e+04	6.5e+07/**1.8e+07**
Compiler	7.1e+08/**5.8e+08**	**9.9e+08**/1.9e+09	1.5e+07/**4.2e+06**	**2.6e+05**/3.3e+05	2.5e+06/**1.1e+05**	**1.6e+04**/3.6e+04	4.6e+07/**7.0e+06**
DBQuery	**9.5e+08**/1.0e+09	**1.1e+09**/2.2e+09	**4.0e+06**/4.0e+06	**1.7e+05**/3.5e+05	8.1e+05/**1.1e+05**	**8.1e+03**/2.2e+04	2.1e+07/**3.5e+06**
DecisionTree	1.8e+10/**1.6e+10**	**4.0e+10**/6.8e+10	5.3e+08/**3.0e+07**	8.4e+04/**2.6e+04**	2.2e+06/**2.1e+06**	8.3e+04/**2.5e+04**	1.1e+09/**3.0e+07**
DomTree	8.3e+08/**5.5e+08**	**1.2e+09**/2.2e+09	1.2e+07/**7.3e+06**	**1.0e+05**/1.2e+05	1.1e+06/**7.8e+04**	**7.2e+03**/3.1e+04	5.7e+07/**5.6e+06**
KDTree	9.1e+08/**7.1e+08**	**3.1e+09**/4.0e+09	1.3e+07/**1.2e+07**	**1.8e+03**/2.0e+03	**2.9e+04**/1.1e+05	**1.5e+03**/1.9e+03	4.4e+07/**9.9e+06**
LinearListReduction	1.5e+08/**2.1e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.3e+04**	5.8e+02/**3.5e+02**	2.8e+05/**7.0e+03**	4.4e+02/**2.2e+02**	1.2e+07/**1.1e+06**
List	**1.2e+09**/1.5e+09	**4.5e+09**/5.6e+09	9.4e+06/**2.4e+06**	**2.5e+05**/5.7e+05	**2.1e+04**/1.9e+05	**7.0e+03**/9.8e+03	4.9e+07/**3.3e+07**
MonoTree	**2.5e+08**/2.6e+08	**9.1e+08**/1.1e+09	**2.4e+05**/4.9e+05	**3.8e+04**/5.7e+04	**4.0e+03**/1.3e+04	**2.5e+03**/4.5e+03	2.2e+06/**1.8e+06**
ObjectGraph	**8.4e+08**/8.7e+08	**3.0e+09**/4.0e+09	9.7e+06/**3.5e+06**	**1.9e+05**/2.5e+05	**3.1e+04**/8.9e+04	**1.0e+04**/1.5e+04	2.4e+07/**8.1e+06**
PiecewiseFunctions	1.2e+09/**1.1e+09**	**2.8e+09**/4.7e+09	1.4e+07/**4.8e+06**	**4.3e+05**/4.5e+05	2.1e+05/**9.2e+04**	**1.6e+04**/3.8e+04	4.5e+07/**1.0e+07**
TernaryTree	4.1e+08/**3.6e+08**	**1.2e+09**/1.5e+09	**4.8e+05**/1.3e+06	**7.0e+04**/1.3e+05	**4.4e+03**/2.0e+04	4.0e+03/**3.5e+03**	3.3e+06/**2.3e+06**
Trie	6.6e+08/**5.7e+08**	**1.2e+09**/2.2e+09	8.6e+06/**3.0e+06**	**1.9e+05**/2.2e+05	1.6e+05/**5.4e+04**	**7.4e+03**/3.6e+04	2.6e+07/**4.6e+06**

Table 4: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**AoS med±err**	**SoA med±err**	**Speedup**
barnesHutPotential	F	11/16	31%	0.0411	0.0396	1.04×
clearFlags	M	15/16	6%	**0.1540**	0.1777	0.87×
countActive	F	10/16	38%	0.0351	0.0344	1.02×
countParticles	F	8/16	50%	0.0362	**0.0316**	**1.14**×
fmmPotential	F	12/16	25%	0.0975	0.1031	0.95×
scaleEnergy	M	16/16	0%	**0.1548**	0.1768	0.88×
sumEnergy	F	12/16	25%	0.0359	0.0341	1.05×
sumMass	F	10/16	38%	0.0359	**0.0291**	**1.23**×
paletteEntriesQuantized	F	13/25	48%	0.0467	**0.0356**	**1.31**×
quantizationErrorProxy	F	10/25	60%	0.0491	**0.0429**	**1.14**×
Total				0.6862	0.7049	0.97×
Geomean						1.05×

Table 5: Per-pass PAPI counters for `OctTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
barnesHutPotential	F	2.1e+08/**2.0e+08**	**8.2e+08**/1.0e+09	**6.8e+04**/5.6e+05	5.7e+02/**5.3e+02**	**1.8e+03**/2.3e+04	3.8e+02/**3.7e+02**	6.2e+06/**2.3e+06**
clearFlags	M	**3.7e+08**/4.4e+08	**8.0e+08**/1.7e+09	**8.2e+05**/2.2e+06	**2.2e+05**/4.2e+05	**8.8e+03**/3.6e+04	**5.7e+03**/1.1e+04	5.6e+06/**2.4e+06**
countActive	F	1.8e+08/**1.6e+08**	5.8e+08/**5.8e+08**	**9.1e+05**/9.1e+05	5.8e+02/**4.8e+02**	**4.3e+03**/2.3e+04	4.8e+02/**2.8e+02**	6.3e+06/**3.0e+05**
countParticles	F	1.8e+08/**1.5e+08**	5.7e+08/**5.7e+08**	9.8e+05/**3.1e+05**	5.8e+02/**4.5e+02**	**3.8e+03**/7.9e+03	4.6e+02/**3.0e+02**	6.1e+06/**1.1e+05**
fmmPotential	F	**4.9e+08**/5.1e+08	**1.6e+09**/1.8e+09	**2.5e+04**/7.6e+06	**5.2e+02**/1.1e+03	**1.6e+03**/1.4e+05	**3.9e+02**/9.4e+02	4.5e+06/**2.0e+06**
scaleEnergy	M	**3.5e+08**/4.4e+08	**9.1e+08**/1.9e+09	**1.3e+06**/2.6e+06	**1.6e+05**/3.9e+05	**7.9e+03**/3.9e+04	**5.6e+03**/1.0e+04	5.6e+06/**2.5e+06**
sumEnergy	F	1.8e+08/**1.7e+08**	**7.7e+08**/8.9e+08	**3.5e+05**/1.1e+06	5.7e+02/**4.8e+02**	**2.2e+03**/3.3e+04	4.0e+02/**3.2e+02**	6.4e+06/**2.7e+06**
sumMass	F	1.8e+08/**1.4e+08**	**5.7e+08**/6.3e+08	7.7e+05/**1.7e+05**	5.6e+02/**4.3e+02**	**3.0e+03**/8.4e+03	4.5e+02/**2.6e+02**	6.2e+06/**1.3e+06**
paletteEntriesQuantized	F	2.3e+08/**1.7e+08**	**6.7e+08**/7.6e+08	4.8e+06/**1.1e+06**	5.9e+02/**5.3e+02**	**1.3e+04**/4.1e+04	4.8e+02/**3.5e+02**	9.4e+06/**1.4e+06**
quantizationErrorProxy	F	2.4e+08/**2.2e+08**	**9.7e+08**/1.2e+09	4.2e+06/**1.3e+06**	4.8e+02/**3.3e+02**	**1.7e+04**/3.4e+04	4.7e+02/**3.1e+02**	9.2e+06/**2.9e+06**
Total		2.6e+09/2.6e+09	8.2e+09/1.1e+10	1.4e+07/1.8e+07	3.8e+05/8.1e+05	6.3e+04/3.9e+05	1.5e+04/2.4e+04	6.5e+07/1.8e+07

Table 6: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
blockCountPass	F	2/9	78%	0.0361	**0.0037**	**9.79**×
branchStatsPass	F	2/9	78%	0.0359	**0.0047**	**7.68**×
castInstCountPass	F	3/9	67%	0.0719	**0.0021**	**33.91**×
has cycle	F	4/9	56%	0.0752	**0.0332**	**2.27**×
instCountPass	F	2/9	78%	0.0342	**0.0043**	**7.90**×
latencyModelPass	F	3/9	67%	0.0362	**0.0035**	**10.26**×
memoryOpStatsPass	F	3/9	67%	0.0360	**0.0058**	**6.25**×
stripSideEffectsPass	M	7/9	22%	**0.0801**	0.0913	0.88×
targetReturnPass	M	9/9	0%	0.0819	0.0880	0.93×
throughputModelPass	F	3/9	67%	0.0122	**0.0029**	**4.15**×
verifyIR	F	9/9	0%	0.0178	**0.0073**	**2.44**×
Total				0.5176	0.2469	**2.10**×
Geomean						**4.69**×

Table 7: Per-pass PAPI counters for `Compiler` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
blockCountPass	F	2.8e+07/**9.8e+06**	5.1e+07/**4.5e+07**	1.6e+06/**6.6e+03**	6.5e+02/**8.7e+01**	5.2e+05/**1.1e+03**	6.1e+02/**7.3e+01**	4.1e+06/**1.6e+04**
branchStatsPass	F	2.8e+07/**1.6e+07**	**7.9e+07**/8.6e+07	8.8e+05/**1.9e+04**	1.0e+03/**1.0e+02**	1.4e+05/**3.4e+03**	9.6e+02/**9.6e+01**	4.3e+06/**4.9e+05**
castInstCountPass	F	6.3e+07/**6.6e+06**	5.1e+07/**3.9e+07**	5.6e+05/**3.2e+04**	6.0e+02/**8.1e+01**	9.7e+03/**1.1e+03**	5.2e+02/**6.6e+01**	2.6e+06/**1.6e+04**
has cycle	F	9.7e+07/**8.9e+07**	**1.1e+08**/1.5e+08	2.4e+06/**5.2e+05**	6.3e+04/**5.9e+04**	1.8e+05/**2.2e+04**	2.4e+03/**6.1e+02**	4.8e+06/**1.5e+06**
instCountPass	F	2.9e+07/**1.1e+07**	**5.6e+07**/5.7e+07	1.6e+06/**8.2e+04**	5.6e+02/**1.4e+02**	5.0e+05/**2.3e+03**	5.2e+02/**1.2e+02**	4.1e+06/**3.8e+04**
latencyModelPass	F	2.9e+07/**1.3e+07**	**6.2e+07**/6.9e+07	1.7e+06/**3.5e+04**	8.6e+02/**1.3e+02**	5.7e+05/**3.9e+03**	8.2e+02/**1.0e+02**	4.2e+06/**6.2e+05**
memoryOpStatsPass	F	2.8e+07/**1.7e+07**	**8.5e+07**/9.1e+07	5.8e+05/**2.4e+04**	6.5e+02/**1.2e+02**	4.7e+04/**3.3e+03**	6.1e+02/**1.1e+02**	4.4e+06/**4.9e+05**
stripSideEffectsPass	M	**1.5e+08**/2.0e+08	**1.8e+08**/6.0e+08	1.6e+06/**1.2e+06**	**1.0e+05**/1.4e+05	**6.7e+03**/1.9e+04	**2.9e+03**/1.2e+04	4.3e+06/**1.2e+06**
targetReturnPass	M	**1.5e+08**/1.9e+08	**1.7e+08**/6.3e+08	1.5e+06/**1.4e+06**	**9.1e+04**/1.3e+05	**5.6e+03**/2.4e+04	**4.0e+03**/2.3e+04	3.7e+06/**1.5e+06**
throughputModelPass	F	6.1e+07/**1.5e+07**	**6.2e+07**/6.9e+07	1.2e+06/**3.2e+04**	8.3e+02/**1.1e+02**	2.1e+05/**3.6e+03**	7.8e+02/**8.3e+01**	4.8e+06/**6.9e+05**
verifyIR	F	4.2e+07/**1.7e+07**	**7.4e+07**/8.6e+07	1.2e+06/**7.8e+05**	1.6e+03/**6.6e+02**	2.9e+05/**2.8e+04**	1.5e+03/**5.2e+02**	4.7e+06/**4.2e+05**
Total		7.1e+08/5.8e+08	9.9e+08/1.9e+09	1.5e+07/4.2e+06	2.6e+05/3.3e+05	2.5e+06/1.1e+05	1.6e+04/3.6e+04	4.6e+07/7.0e+06

Table 8: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
clearQueryFlags	M	14/15	7%	**0.0692**	0.1053	0.66×
countJoins	F	3/15	80%	0.0196	**0.0116**	**1.69**×
filterSelectivitySkew	F	5/15	67%	0.0194	**0.0125**	**1.55**×
hashJoinPressure	F	5/15	67%	0.0198	**0.0129**	**1.53**×
scaleCosts	M	15/15	0%	**0.0715**	0.0847	0.84×
sumCost	F	6/15	60%	**0.0195**	0.0222	0.88×
sumMemory	F	6/15	60%	0.0192	**0.0154**	**1.25**×
sumRows	F	6/15	60%	0.0264	**0.0226**	**1.16**×
Total				0.2645	0.2872	0.92×
Geomean						**1.14**×

Table 9: Per-pass PAPI counters for `DBQuery` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
clearQueryFlags	M	**1.6e+08**/3.3e+08	**1.9e+08**/7.2e+08	**3.1e+05**/1.5e+06	**8.9e+04**/1.6e+05	3.0e+04/**2.4e+04**	**2.6e+03**/9.5e+03	2.3e+06/**8.9e+05**
countJoins	F	9.9e+07/**5.9e+07**	7.4e+07/**7.1e+07**	6.2e+05/**2.5e+04**	5.4e+02/**2.3e+02**	1.4e+05/**2.0e+03**	4.9e+02/**1.6e+02**	2.7e+06/**5.8e+04**
filterSelectivitySkew	F	9.8e+07/**6.3e+07**	**1.0e+08**/1.1e+08	5.1e+05/**7.8e+04**	2.1e+02/**1.4e+02**	1.2e+05/**6.5e+03**	1.8e+02/**1.2e+02**	2.7e+06/**3.2e+05**
hashJoinPressure	F	1.0e+08/**6.5e+07**	**8.4e+07**/9.9e+07	7.3e+05/**1.0e+05**	3.1e+02/**1.7e+02**	1.9e+05/**7.4e+03**	3.0e+02/**1.4e+02**	2.8e+06/**1.3e+05**
scaleCosts	M	**1.6e+08**/2.2e+08	**2.2e+08**/7.5e+08	**3.1e+05**/1.2e+06	**7.5e+04**/1.8e+05	3.6e+04/**2.7e+04**	**3.7e+03**/1.2e+04	2.5e+06/**1.1e+06**
sumCost	F	9.9e+07/**7.7e+07**	**9.0e+07**/1.2e+08	**5.6e+05**/9.3e+05	5.3e+02/**3.1e+02**	1.2e+05/**2.2e+04**	4.2e+02/**2.5e+02**	2.7e+06/**3.0e+05**
sumMemory	F	9.7e+07/**7.8e+07**	**9.0e+07**/1.2e+08	6.8e+05/**8.1e+04**	2.6e+02/**1.1e+02**	1.6e+05/**1.2e+04**	2.3e+02/**1.0e+02**	2.8e+06/**3.7e+05**
sumRows	F	1.3e+08/**1.1e+08**	**2.1e+08**/2.3e+08	2.6e+05/**1.2e+05**	2.4e+02/**1.8e+02**	2.2e+04/**7.8e+03**	2.2e+02/**1.7e+02**	2.4e+06/**3.1e+05**
Total		9.5e+08/1.0e+09	1.1e+09/2.2e+09	4.0e+06/4.0e+06	1.7e+05/3.5e+05	8.1e+05/1.1e+05	8.1e+03/2.2e+04	2.1e+07/3.5e+06

Table 10: Per-pass performance for `DecisionTree`, ADT has 7 fields, SoA uses 6 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
classify Batch	F	5/7	29%	3.113	**2.756**	**1.13**×
classify Depth	F	4/7	43%	0.0447	**0.0391**	**1.14**×
classify tree	F	5/7	29%	0.0446	**0.0390**	**1.14**×
countFeatureUses	F	3/7	57%	0.0478	**0.0317**	**1.51**×
countLeaves	F	2/7	71%	0.0457	**0.0251**	**1.83**×
countNodes	F	2/7	71%	0.0461	**0.0281**	**1.64**×
countSmallLeaves	F	3/7	57%	0.0486	**0.0320**	**1.52**×
inferenceCost	F	2/7	71%	0.0483	**0.0290**	**1.66**×
sumImpurity	F	3/7	57%	0.0479	**0.0321**	**1.49**×
sumPathLengths	F	3/7	57%	0.0509	**0.0333**	**1.53**×
sumSamples	F	3/7	57%	0.0458	**0.0316**	**1.45**×
treeDepth	F	2/7	71%	0.0486	**0.0280**	**1.74**×
Total				3.632	3.105	**1.17**×
Geomean						**1.46**×

Table 11: Per-pass PAPI counters for `DecisionTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
classify Batch	F	1.6e+10/**1.4e+10**	**3.3e+10**/5.9e+10	4.5e+08/**2.6e+07**	7.1e+04/**1.9e+04**	**1.8e+06**/2.0e+06	7.0e+04/**1.8e+04**	9.3e+08/**1.8e+07**
classify Depth	F	2.3e+08/**2.0e+08**	**4.7e+08**/8.5e+08	6.4e+06/**4.1e+05**	1.1e+03/**3.8e+02**	**2.1e+04**/2.7e+04	1.0e+03/**3.5e+02**	1.3e+07/**3.3e+05**
classify tree	F	2.3e+08/**2.0e+08**	**4.7e+08**/8.5e+08	6.4e+06/**4.1e+05**	1.1e+03/**2.7e+02**	**2.1e+04**/2.8e+04	1.1e+03/**2.2e+02**	1.3e+07/**3.4e+05**
countFeatureUses	F	2.4e+08/**1.6e+08**	**7.5e+08**/7.8e+08	7.6e+06/**2.9e+05**	7.9e+02/**7.4e+02**	3.7e+04/**1.4e+04**	7.7e+02/**7.2e+02**	1.5e+07/**2.1e+06**
countLeaves	F	2.3e+08/**1.3e+08**	**6.0e+08**/6.1e+08	7.2e+06/**9.7e+04**	1.1e+03/**5.6e+02**	4.4e+04/**7.9e+03**	1.1e+03/**5.3e+02**	1.4e+07/**4.1e+05**
countNodes	F	2.3e+08/**1.3e+08**	**6.1e+08**/6.2e+08	6.9e+06/**3.8e+05**	1.6e+03/**7.6e+02**	5.2e+04/**1.2e+04**	1.4e+03/**6.6e+02**	1.4e+07/**3.5e+05**
countSmallLeaves	F	2.4e+08/**1.6e+08**	**7.1e+08**/8.4e+08	5.9e+06/**8.5e+04**	**7.8e+02**/8.2e+02	3.5e+04/**1.3e+04**	**7.6e+02**/8.0e+02	1.5e+07/**2.0e+06**
inferenceCost	F	2.5e+08/**1.5e+08**	6.1e+08/**6.1e+08**	7.9e+06/**1.2e+05**	1.1e+03/**8.2e+02**	1.7e+04/**8.1e+03**	1.1e+03/**7.7e+02**	1.4e+07/**4.2e+05**
sumImpurity	F	2.4e+08/**1.6e+08**	**7.6e+08**/7.8e+08	7.7e+06/**1.1e+06**	1.4e+03/**6.4e+02**	3.6e+04/**1.3e+04**	1.4e+03/**6.1e+02**	1.4e+07/**2.1e+06**
sumPathLengths	F	2.5e+08/**1.7e+08**	**8.0e+08**/9.0e+08	7.7e+06/**4.8e+05**	1.7e+03/**6.8e+02**	3.9e+04/**1.4e+04**	1.6e+03/**6.6e+02**	1.5e+07/**2.0e+06**
sumSamples	F	2.3e+08/**1.6e+08**	**6.4e+08**/7.7e+08	6.1e+06/**2.4e+05**	7.7e+02/**7.2e+02**	4.3e+04/**1.4e+04**	7.5e+02/**6.8e+02**	1.5e+07/**2.1e+06**
treeDepth	F	2.5e+08/**1.4e+08**	6.1e+08/**6.1e+08**	8.0e+06/**1.1e+05**	1.1e+03/**7.3e+02**	1.8e+04/**8.2e+03**	1.1e+03/**6.8e+02**	1.4e+07/**4.2e+05**
Total		1.8e+10/1.6e+10	4.0e+10/6.8e+10	5.3e+08/3.0e+07	8.4e+04/2.6e+04	2.2e+06/2.1e+06	8.3e+04/2.5e+04	1.1e+09/3.0e+07

Table 12: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
SumArea	F	6/15	60%	0.0365	**0.0214**	**1.71**×
computeWidths	M	14/15	7%	**0.0300**	0.0445	0.67×
count styled	F	3/15	80%	0.0346	**0.0117**	**2.95**×
find max Bottom	F	5/15	67%	0.0346	**0.0185**	**1.87**×
scaleLayout	M	15/15	0%	**0.0288**	0.0371	0.78×
sumTextWidth	F	3/15	80%	0.0344	**0.0118**	**2.92**×
Total				0.1989	0.1450	**1.37**×
Geomean						**1.56**×

Table 13: Per-pass PAPI counters for `DomTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
SumArea	F	1.8e+08/**1.0e+08**	**2.9e+08**/4.9e+08	2.5e+06/**2.0e+06**	8.0e+02/**4.9e+02**	1.6e+05/**4.0e+04**	6.9e+02/**3.4e+02**	1.4e+07/**1.8e+06**
computeWidths	M	**6.5e+07**/1.4e+08	**1.4e+08**/4.6e+08	**1.2e+06**/1.6e+06	**5.2e+04**/6.9e+04	**4.4e+03**/1.0e+04	**1.7e+03**/1.5e+04	1.3e+06/**4.5e+05**
count styled	F	1.8e+08/**5.9e+07**	**2.6e+08**/2.6e+08	2.1e+06/**2.5e+04**	1.3e+03/**1.2e+02**	3.0e+05/**4.9e+03**	1.0e+03/**1.0e+02**	1.4e+07/**7.2e+05**
find max Bottom	F	1.7e+08/**9.4e+07**	**2.5e+08**/4.1e+08	2.4e+06/**1.4e+06**	4.6e+02/**1.5e+02**	1.6e+05/**9.5e+03**	4.3e+02/**1.2e+02**	1.4e+07/**1.5e+06**
scaleLayout	M	**5.9e+07**/1.0e+08	**8.8e+07**/2.9e+08	**1.1e+06**/2.3e+06	**4.4e+04**/5.0e+04	2.3e+04/**9.2e+03**	**2.7e+03**/1.5e+04	1.4e+06/**4.5e+05**
sumTextWidth	F	1.7e+08/**6.0e+07**	**2.0e+08**/2.4e+08	2.5e+06/**1.9e+04**	7.7e+02/**2.0e+02**	4.7e+05/**4.4e+03**	7.2e+02/**1.9e+02**	1.4e+07/**7.2e+05**
Total		8.3e+08/5.5e+08	1.2e+09/2.2e+09	1.2e+07/7.3e+06	1.0e+05/1.2e+05	1.1e+06/7.8e+04	7.2e+03/3.1e+04	5.7e+07/5.6e+06

Table 14: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
Find nearest Neighbour	F	13/17	24%	0.0281	**0.0238**	**1.18**×
countInRange tight_box	F	13/17	24%	0.0277	**0.0198**	**1.40**×
photonMappingBenchmark	F	12/17	29%	0.0427	0.0421	1.01×
pointCloudNeighborhood	F	11/17	35%	0.0248	**0.0165**	**1.51**×
sumMassInRange	F	14/17	18%	0.0271	**0.0202**	**1.34**×
twoPointCorrelation bin_8_16	F	11/17	35%	0.0285	**0.0207**	**1.37**×
Total				0.1788	0.1431	**1.25**×
Geomean						**1.29**×

Table 15: Per-pass PAPI counters for `KDTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
Find nearest Neighbour	F	1.4e+08/**1.1e+08**	**4.7e+08**/6.3e+08	2.7e+06/**2.0e+06**	6.1e+02/**4.7e+02**	**4.7e+03**/2.5e+04	4.4e+02/**3.7e+02**	7.8e+06/**1.7e+06**
countInRange tight_box	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.5e+08	2.0e+06/**1.8e+06**	2.3e+02/**1.5e+02**	**2.8e+03**/1.7e+04	2.1e+02/**1.6e+02**	7.8e+06/**1.8e+06**
photonMappingBenchmark	F	2.2e+08/**2.1e+08**	**1.0e+09**/1.2e+09	**2.1e+06**/2.5e+06	**3.9e+02**/8.4e+02	**1.1e+04**/1.8e+04	**3.6e+02**/8.3e+02	5.6e+06/**1.5e+06**
pointCloudNeighborhood	F	1.3e+08/**8.3e+07**	**3.3e+08**/4.8e+08	2.4e+06/**2.0e+06**	1.7e+02/1.7e+02	**4.8e+03**/1.6e+04	1.5e+02/**1.5e+02**	7.8e+06/**1.5e+06**
sumMassInRange	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.7e+08	**2.1e+06**/2.9e+06	2.1e+02/**1.9e+02**	**3.6e+03**/2.0e+04	**1.8e+02**/1.9e+02	7.7e+06/**2.0e+06**
twoPointCorrelation bin_8_16	F	1.4e+08/**1.1e+08**	**4.8e+08**/6.1e+08	2.1e+06/**7.2e+05**	**1.7e+02**/1.8e+02	**2.8e+03**/1.3e+04	**1.7e+02**/1.8e+02	7.7e+06/**1.4e+06**
Total		9.1e+08/7.1e+08	3.1e+09/4.0e+09	1.3e+07/1.2e+07	1.8e+03/2.0e+03	2.9e+04/1.1e+05	1.5e+03/1.9e+03	4.4e+07/9.9e+06

Table 16: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
reduction	F	2/11	82%	0.0295	**0.0062**	**4.74**×
Total				0.0295	0.0062	**4.74**×
Geomean						**4.74**×

Table 17: Per-pass PAPI counters for `LinearListReduction` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
reduction	F	1.5e+08/**2.1e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.3e+04**	5.8e+02/**3.5e+02**	2.8e+05/**7.0e+03**	4.4e+02/**2.2e+02**	1.2e+07/**1.1e+06**
Total		1.5e+08/2.1e+07	9.0e+07/1.0e+08	1.5e+06/4.3e+04	5.8e+02/3.5e+02	2.8e+05/7.0e+03	4.4e+02/2.2e+02	1.2e+07/1.1e+06

Table 18: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1 List	M	2/2	0%	**0.2541**	0.3404	0.75×
length List	F	1/2	50%	0.0412	**0.0215**	**1.91**×
sumList	F	2/2	0%	0.0441	0.0452	0.98×
sumList tail recursive	F	2/2	0%	0.0443	0.0446	0.99×
Total				0.3837	0.4517	0.85×
Geomean						1.08×

Table 19: Per-pass PAPI counters for `List` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1 List	M	**5.9e+08**/9.7e+08	**1.9e+09**/2.8e+09	8.0e+06/**1.1e+06**	**2.5e+05**/5.7e+05	**1.1e+04**/5.2e+04	**6.5e+03**/9.4e+03	9.8e+06/**7.8e+06**
length List	F	2.1e+08/**1.1e+08**	8.0e+08/**8.0e+08**	1.1e+05/**1.1e+05**	1.7e+02/**1.1e+02**	**2.9e+03**/1.4e+04	1.6e+02/**9.4e+01**	1.3e+07/**1.2e+06**
sumList	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	6.8e+05/**5.4e+05**	1.5e+02/**1.3e+02**	**3.4e+03**/6.7e+04	1.2e+02/**1.2e+02**	1.3e+07/**1.2e+07**
sumList tail recursive	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	5.8e+05/**5.7e+05**	**1.9e+02**/2.0e+02	**3.3e+03**/5.8e+04	1.8e+02/**1.8e+02**	1.3e+07/**1.2e+07**
Total		1.2e+09/1.5e+09	4.5e+09/5.6e+09	9.4e+06/2.4e+06	2.5e+05/5.7e+05	2.1e+04/1.9e+05	7.0e+03/9.8e+03	4.9e+07/3.3e+07

Table 20: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add1Tree	M	3/3	0%	**0.0529**	0.0782	0.68×
sumTree	F	3/3	0%	**0.0092**	0.0115	0.80×
sumTree TailRec	F	3/3	0%	**0.0085**	0.0111	0.77×
Total				0.0706	0.1007	0.70×
Geomean						0.75×

Table 21: Per-pass PAPI counters for `MonoTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1Tree	M	1.6e+08/**1.5e+08**	**5.0e+08**/5.9e+08	**2.3e+05**/4.4e+05	**3.8e+04**/5.7e+04	**2.6e+03**/5.1e+03	**2.0e+03**/3.8e+03	5.0e+05/**4.4e+05**
sumTree	F	**4.6e+07**/5.8e+07	**2.2e+08**/2.6e+08	**7.0e+03**/3.5e+04	**3.1e+02**/3.6e+02	**6.9e+02**/3.9e+03	**2.6e+02**/3.2e+02	8.7e+05/**6.9e+05**
sumTree TailRec	F	**4.3e+07**/5.6e+07	**1.9e+08**/2.2e+08	**6.2e+03**/1.8e+04	**3.0e+02**/4.7e+02	**6.8e+02**/3.9e+03	**2.5e+02**/4.4e+02	8.7e+05/**6.9e+05**
Total		2.5e+08/2.6e+08	9.1e+08/1.1e+09	2.4e+05/4.9e+05	3.8e+04/5.7e+04	4.0e+03/1.3e+04	2.5e+03/4.5e+03	2.2e+06/1.8e+06

Table 22: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
countLargeItems	F	3/5	40%	0.0126	0.0139	0.91×
countMarked	F	3/5	40%	**0.0128**	0.0232	0.55×
countSurvivors	F	4/5	20%	0.0136	0.0129	1.06×
deadBytes	F	4/5	20%	0.0133	0.0123	1.08×
liveBytes	F	4/5	20%	0.0132	0.0124	1.06×
sumObjIds	F	3/5	40%	0.0126	**0.0107**	**1.18**×
sweepUnmarked	M	5/5	0%	**0.0718**	0.0804	0.89×
totalHeapSize	F	3/5	40%	**0.0126**	0.0386	0.33×
touchHotObjects	M	5/5	0%	**0.0711**	0.0818	0.87×
Total				0.2336	0.2862	0.82×
Geomean						0.83×

Table 23: Per-pass PAPI counters for `ObjectGraph` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
countLargeItems	F	6.4e+07/**5.1e+07**	**2.6e+08**/2.8e+08	1.2e+06/**6.4e+04**	4.3e+02/**4.2e+02**	**3.4e+03**/3.9e+03	3.9e+02/**3.9e+02**	2.9e+06/**6.7e+05**
countMarked	F	6.4e+07/**5.4e+07**	**2.6e+08**/2.8e+08	1.2e+06/**5.9e+05**	5.2e+02/**4.6e+02**	**3.3e+03**/1.4e+04	4.7e+02/**4.3e+02**	2.9e+06/**5.3e+05**
countSurvivors	F	6.9e+07/**6.5e+07**	**3.1e+08**/3.8e+08	9.5e+05/**7.1e+04**	3.6e+02/**3.3e+02**	**2.4e+03**/5.3e+03	**3.3e+02**/3.3e+02	2.8e+06/**1.3e+06**
deadBytes	F	6.7e+07/**6.3e+07**	**2.7e+08**/3.5e+08	7.4e+05/**5.5e+04**	**2.6e+02**/3.0e+02	**2.6e+03**/5.6e+03	**2.4e+02**/3.0e+02	2.8e+06/**1.3e+06**
liveBytes	F	6.6e+07/**6.2e+07**	**2.7e+08**/3.5e+08	9.4e+05/**5.3e+04**	3.1e+02/**3.1e+02**	**2.5e+03**/5.6e+03	**2.8e+02**/2.9e+02	2.8e+06/**1.3e+06**
sumObjIds	F	6.4e+07/**5.4e+07**	**2.5e+08**/2.7e+08	1.2e+06/**7.3e+04**	3.5e+02/**3.0e+02**	**2.9e+03**/4.4e+03	3.3e+02/**2.8e+02**	2.8e+06/**6.8e+05**
sweepUnmarked	M	**1.9e+08**/2.4e+08	**5.5e+08**/8.8e+08	1.1e+06/**4.7e+05**	**7.5e+04**/1.1e+05	**6.0e+03**/1.1e+04	**4.0e+03**/6.4e+03	1.9e+06/**9.9e+05**
totalHeapSize	F	6.4e+07/**5.3e+07**	**2.5e+08**/2.7e+08	**1.2e+06**/1.3e+06	**6.5e+02**/8.5e+02	**2.7e+03**/2.6e+04	**5.1e+02**/7.2e+02	2.9e+06/**3.7e+05**
touchHotObjects	M	**1.9e+08**/2.3e+08	**5.7e+08**/9.0e+08	1.2e+06/**8.7e+05**	**1.1e+05**/1.3e+05	**4.9e+03**/1.4e+04	**3.7e+03**/6.2e+03	1.8e+06/**1.0e+06**
Total		8.4e+08/8.7e+08	3.0e+09/4.0e+09	9.7e+06/3.5e+06	1.9e+05/2.5e+05	3.1e+04/8.9e+04	1.0e+04/1.5e+04	2.4e+07/8.1e+06

Table 24: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
addConstPW	M	8/8	0%	0.1190	0.1321	0.90×
autorefineMaxLevel	F	4/8	50%	0.0204	0.0190	1.08×
compressMass	F	3/8	62%	0.0194	**0.0115**	**1.69**×
diffPW	M	8/8	0%	0.1206	0.1285	0.94×
lbDeuxLoadProxy	F	5/8	38%	0.0211	**0.0183**	**1.16**×
norm2Estimate	F	5/8	38%	0.0255	0.0234	1.09×
pmapCutHistogram	F	4/8	50%	0.0318	**0.0124**	**2.55**×
truncateTolViolations	F	3/8	62%	0.0200	**0.0116**	**1.73**×
Total				0.3778	0.3567	1.06×
Geomean						**1.31**×

Table 25: Per-pass PAPI counters for `PiecewiseFunctions` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
addConstPW	M	**2.7e+08**/3.3e+08	**5.7e+08**/1.2e+09	**1.6e+06**/1.6e+06	2.9e+05/**2.6e+05**	**1.3e+04**/2.3e+04	**5.0e+03**/1.7e+04	4.6e+06/**1.8e+06**
autorefineMaxLevel	F	1.0e+08/**9.6e+07**	**2.5e+08**/3.7e+08	2.5e+06/**1.9e+05**	8.0e+02/**1.3e+02**	1.7e+04/**6.3e+03**	7.8e+02/**1.1e+02**	6.1e+06/**9.9e+05**
compressMass	F	9.8e+07/**5.8e+07**	**2.5e+08**/2.8e+08	2.1e+06/**2.8e+04**	7.4e+02/**4.0e+02**	2.1e+04/**5.1e+03**	7.3e+02/**3.8e+02**	6.3e+06/**7.0e+05**
diffPW	M	**2.8e+08**/3.2e+08	**5.9e+08**/1.3e+09	**8.6e+05**/1.9e+06	**1.4e+05**/1.9e+05	**1.0e+04**/2.3e+04	**6.5e+03**/1.9e+04	4.1e+06/**1.8e+06**
lbDeuxLoadProxy	F	1.1e+08/**9.3e+07**	**3.3e+08**/4.9e+08	2.4e+06/**3.0e+05**	6.6e+02/**2.2e+02**	1.5e+04/**9.0e+03**	6.4e+02/**2.0e+02**	6.3e+06/**1.6e+06**
norm2Estimate	F	1.3e+08/**8.9e+07**	**3.4e+08**/4.4e+08	1.1e+06/**6.8e+05**	8.3e+02/**4.9e+02**	**1.3e+04**/1.4e+04	7.2e+02/**3.9e+02**	6.1e+06/**1.2e+06**
pmapCutHistogram	F	1.6e+08/**6.3e+07**	**2.8e+08**/3.7e+08	1.7e+06/**5.6e+04**	1.3e+03/**4.1e+02**	1.0e+05/**7.3e+03**	1.3e+03/**3.8e+02**	4.8e+06/**1.3e+06**
truncateTolViolations	F	1.0e+08/**5.8e+07**	**2.5e+08**/2.9e+08	2.0e+06/**3.0e+04**	9.0e+02/**3.4e+02**	1.8e+04/**4.8e+03**	8.6e+02/**2.8e+02**	6.3e+06/**7.1e+05**
Total		1.2e+09/1.1e+09	2.8e+09/4.7e+09	1.4e+07/4.8e+06	4.3e+05/4.5e+05	2.1e+05/9.2e+04	1.6e+04/3.8e+04	4.5e+07/1.0e+07

Table 26: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
add 1 tree	M	5/5	0%	0.0834	0.0892	0.94×
sum tree	F	5/5	0%	0.0280	0.0261	1.07×
Total				0.1113	0.1152	0.97×
Geomean						1.00×

Table 27: Per-pass PAPI counters for `TernaryTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add 1 tree	M	2.7e+08/**2.3e+08**	**7.5e+08**/9.8e+08	**3.9e+05**/9.8e+05	**7.0e+04**/1.3e+05	**3.6e+03**/1.1e+04	3.9e+03/**3.4e+03**	1.5e+06/**1.1e+06**
sum tree	F	1.4e+08/**1.3e+08**	**4.6e+08**/5.4e+08	**9.6e+04**/3.3e+05	1.2e+02/**1.0e+02**	**8.0e+02**/8.9e+03	1.1e+02/**9.0e+01**	1.8e+06/**1.2e+06**
Total		4.1e+08/3.6e+08	1.2e+09/1.5e+09	4.8e+05/1.3e+06	7.0e+04/1.3e+05	4.4e+03/2.0e+04	4.0e+03/3.5e+03	3.3e+06/2.3e+06

Table 28: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med±err	SoA med±err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0152	**0.0080**	**1.91**×
countLazyNodes	F	4/10	60%	0.0130	**0.0066**	**1.97**×
countTerminals	F	3/10	70%	0.0125	**0.0064**	**1.96**×
decayTrieStats	M	10/10	0%	**0.0751**	0.0850	0.88×
resetTraversalState	M	10/10	0%	0.0760	0.0820	0.93×
sumPrefixFreq	F	3/10	70%	0.0127	**0.0080**	**1.59**×
sumSubtreeHints	F	3/10	70%	0.0123	**0.0059**	**2.10**×
Total				0.2169	0.2018	1.07×
Geomean						**1.54**×

Table 29: Per-pass PAPI counters for `Trie` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
autocompleteTopKProxy	F	7.7e+07/**4.0e+07**	**1.3e+08**/2.1e+08	1.5e+06/**5.3e+05**	1.5e+02/**1.0e+02**	5.3e+04/**4.7e+03**	1.3e+02/**8.3e+01**	3.7e+06/**8.5e+05**
countLazyNodes	F	6.6e+07/**3.3e+07**	**1.3e+08**/1.6e+08	1.3e+06/**2.7e+04**	1.8e+02/**1.7e+02**	1.5e+04/**3.3e+03**	1.6e+02/**1.5e+02**	4.0e+06/**6.6e+05**
countTerminals	F	6.3e+07/**2.9e+07**	**1.0e+08**/1.2e+08	1.3e+06/**1.1e+04**	3.8e+02/**1.9e+02**	2.8e+04/**2.2e+03**	3.6e+02/**1.7e+02**	4.0e+06/**3.5e+05**
decayTrieStats	M	**1.6e+08**/2.1e+08	**3.5e+08**/7.9e+08	**7.5e+05**/1.2e+06	**9.4e+04**/1.1e+05	**5.9e+03**/1.9e+04	**2.9e+03**/2.2e+04	2.8e+06/**1.2e+06**
resetTraversalState	M	**1.6e+08**/2.0e+08	**2.8e+08**/7.0e+08	9.6e+05/**7.9e+05**	**9.2e+04**/1.1e+05	**8.1e+03**/1.3e+04	**3.1e+03**/1.3e+04	3.3e+06/**8.9e+05**
sumPrefixFreq	F	6.4e+07/**3.1e+07**	**1.1e+08**/1.2e+08	1.3e+06/**4.2e+05**	5.8e+02/**3.7e+02**	2.7e+04/**9.3e+03**	4.3e+02/**2.5e+02**	4.0e+06/**3.0e+05**
sumSubtreeHints	F	6.2e+07/**3.0e+07**	**1.1e+08**/1.2e+08	1.5e+06/**1.7e+04**	2.8e+02/**1.6e+02**	2.6e+04/**2.5e+03**	2.6e+02/**1.5e+02**	3.9e+06/**3.5e+05**
Total		6.6e+08/5.7e+08	1.2e+09/2.2e+09	8.6e+06/3.0e+06	1.9e+05/2.2e+05	1.6e+05/5.4e+04	7.4e+03/3.6e+04	2.6e+07/4.6e+06